

Laboratorio 9 - Redes Neuronales Artificiales

Bryan Martínez, Adriana Palacios, y Nery Molina

2026-05-10

Preparación de datos y control de comparabilidad

Para garantizar que los resultados encontrados con este modelo que se va a estar generando sean comparables con todos los anteriores, se busca utilizar los mismos conjuntos de entrenamiento y prueba. Utilizando la misma semilla y una partición de 70% para datos de entrenamiento y 30% para datos de prueba, establecido desde el laboratorio 3. Se definió de esta forma porque es la sugerencia que se dio dentro del curso como recomendación para tratar de tener modelos que no sean sobreajustados ni subajustados. Cabe recalcar que, al estar trabajando con redes neuronales, un enfoque en el cual se tiene sensibilidad ante la escala de los valores que se manejan (ya que el aprendizaje se va calculando en base a los gradientes descendentes), diferencias considerables en las magnitudes de las variables pueden afectar directamente a la efectividad del modelo. ya que podría tomar caminos en donde, dadas las magnitudes tan grandes que pueden tener algunas variables, parezca que representa una tasa de cambio mucho más alta, cuando lo único que está ocurriendo es que las unidades con las que se trabajan son mayores para algunas variables. Por ello, para trabajar el resto de laboratorios, se aplicó normalización min-max sobre cada una de las variables predictoras, transformando todos los valores a un rango de entre 0 y 1. Además, para evitar que la información del conjunto de entrenamiento se filtrara hacia el conjunto de prueba, se calcularon los rangos (mínimo y máximo) exclusivamente desde el conjunto de entrenamiento, y esos mismos parámetros se aplican al conjunto de prueba. Por otro lado, para cuando se llegue a la etapa de trabajar con el modelo para clasificación, se dejaron los mismos umbrales definidos en laboratorios anteriores. En donde: - El percentil 33 va a definir las casas económicas. - Entre 33 y 67 tenemos a las intermedias. - Del percentil 67 para arriba vamos a tener a todas las casas caras.

Definición de segmentos de precio

La variable respuesta seleccionada para los modelos de clasificación es `price_cat`, construida a partir del precio por noche y discretizada en tres categorías: Económica (precio ≤ 135 USD), Intermedia ($135 < \text{precio} \leq 252$ USD) y Cara (precio > 252 USD). Esta misma variable se utilizó en todos los laboratorios de clasificación anteriores, lo que permite que podamos comparar contra resultados en modelos previos.

En los conjuntos normalizados para RNA (`train_rna_clf` y `test_rna_clf`), la variable respuesta también está representada mediante tres columnas binarias (Económica, Intermedia, Cara). Dado que las redes neuronales necesitan valores numéricos para poder ponderar sus entradas, las variables categóricas requieren transformarse: para cada valor posible que puede tomar la categoría se crea una columna en el conjunto de datos, y a través de ceros y unos se representa su presencia o ausencia en cada observación.

Arquitecturas de clasificación con redes neuronales

```
set.seed(123)
idx_rna_clf <- sample(nrow(train_rna_clf), min(500, nrow(train_rna_clf)))
```

```
train_rna_sample_clf <- train_rna_clf[idx_rna_clf, ]
```

```
set.seed(123)
tiempo_una_capa <- system.time({
  modelo_una_capa <- neuralnet(
    formula_clf,
    data = train_rna_sample_clf,
    hidden = 64,
    act.fct = "logistic",
    linear.output = FALSE,
    threshold = 0.5,
    stepmax = 1e5
  )
})
```

```
set.seed(123)
tiempo_dos_capas <- system.time({
  modelo_dos_capas <- neuralnet(
    formula_clf,
    data = train_rna_sample_clf,
    hidden = 32,
    act.fct = "tanh",
    linear.output = FALSE,
    threshold = 0.5,
    stepmax = 1e5
  )
})
```

Predicción de segmentos de precio

```
clases_clf <- c("Economica", "Intermedia", "Cara")
```

```
# Predicciones sobre el conjunto de prueba
```

```
pred_probs_una_test <- compute(modelo_una_capa, test_rna_clf[, feature_names_clf])$net.result
pred_clase_una_test <- factor(clases_clf[apply(pred_probs_una_test, 1, which.max)], levels = clases_clf)
```

```
pred_probs_dos_test <- compute(modelo_dos_capas, test_rna_clf[, feature_names_clf])$net.result
pred_clase_dos_test <- factor(clases_clf[apply(pred_probs_dos_test, 1, which.max)], levels = clases_clf)
```

```
# Predicciones sobre entrenamiento (necesarias para análisis de sobreajuste en Actividad 7)
```

```
pred_probs_una_train <- compute(modelo_una_capa, train_rna_clf[, feature_names_clf])$net.result
pred_clase_una_train <- factor(clases_clf[apply(pred_probs_una_train, 1, which.max)], levels = clases_clf)
```

```
pred_probs_dos_train <- compute(modelo_dos_capas, train_rna_clf[, feature_names_clf])$net.result
pred_clase_dos_train <- factor(clases_clf[apply(pred_probs_dos_train, 1, which.max)], levels = clases_clf)
```

Desempeño por matriz de confusión

```

metricas_clf <- function(nombre, pred, real) {
  cat("===", nombre, "===\n\n")
  print(table(Prediccion = pred, Referencia = real))
  cat("\n")

  clases <- c("Economica", "Intermedia", "Cara")
  acc <- mean(as.character(pred) == as.character(real))

  tabla <- t(sapply(clases, function(cls) {
    tp <- sum(pred == cls & real == cls)
    fp <- sum(pred == cls & real != cls)
    fn <- sum(pred != cls & real == cls)
    pr <- ifelse(tp + fp == 0, 0, tp / (tp + fp))
    re <- ifelse(tp + fn == 0, 0, tp / (tp + fn))
    f1 <- ifelse(pr + re == 0, 0, 2 * pr * re / (pr + re))
    c(Precision = round(pr, 4), Recall = round(re, 4), F1 = round(f1, 4))
  })))

  cat("Accuracy:", round(acc, 4), "\n\n")
  print(tabla)
  cat("\n")
}

metricas_clf("Modelo de 64 neuronas", pred_clase_una_test, test_rna_clf$price_cat)

```

```

## === Modelo de 64 neuronas ===
##
##               Referencia
## Prediccion   Economica Intermedia Cara
## Economica      3966      1762  558
## Intermedia     1601      2746 1594
## Cara           618      1854 3967
##
## Accuracy: 0.5721
##
##           Precision Recall    F1
## Economica    0.6309 0.6412 0.6360
## Intermedia    0.4622 0.4316 0.4464
## Cara         0.6161 0.6483 0.6318

```

```

metricas_clf("Modelo de 32 neuronas", pred_clase_dos_test, test_rna_clf$price_cat)

```

```

## === Modelo de 32 neuronas ===
##
##               Referencia
## Prediccion   Economica Intermedia Cara
## Economica      3854      1871  638
## Intermedia     1632      2671 1612
## Cara           699      1820 3869
##
## Accuracy: 0.5568
##

```

##	Precision	Recall	F1
## Economica	0.6057	0.6231	0.6143
## Intermedia	0.4516	0.4198	0.4351
## Cara	0.6057	0.6323	0.6187

Evaluación comparativa de clasificación RNA

```
cat("Tiempo Modelo 64 neuronas: ", tiempo_una_capa["elapsed"], "seg\n")
```

```
## Tiempo Modelo 64 neuronas: 0.489 seg
```

```
cat("Tiempo Modelo 32 neuronas: ", tiempo_dos_capas["elapsed"], "seg\n")
```

```
## Tiempo Modelo 32 neuronas: 1.462 seg
```

Tras analizar los resultados presentados por cada uno de los modelos, se identificó que el modelo de 64 neuronas tiene un accuracy de 0.5721, mientras que el de 32 neuronas alcanzó 0.5568. A pesar de duplicar la cantidad de neuronas, la mejoría entre una métrica y la otra es mínima, de menos de 0.02, lo que puede ser un indicio de que el modelo converge en esa zona. No se logra evidenciar una diferencia notoria a pesar del cambio considerable en la cantidad de neuronas.

Por otro lado, al analizar las matrices de confusión para cada uno de los casos, vemos que donde más se confunde el modelo de 64 neuronas es con la clasificación intermedia de propiedades. Se predijeron como económicas observaciones que resultaron ser intermedias, y como caras otras que también terminaron siendo intermedias. Esto puede explicarse a partir de la idea de que una propiedad intermedia comparte características con una económica y con una cara, lo que le dificulta al modelo hacer una clasificación adecuada para esta categoría.

Un caso similar se observa en el otro modelo, donde una vez más las mayores discrepancias se presentan con la categoría intermedia. Uno de los puntos más destacables al analizar las matrices es que los valores no cambian considerablemente entre un modelo y otro. Por ejemplo, las observaciones predichas como Intermedia que en realidad eran Economica fueron 1,601 en el modelo de 64 neuronas y 1,632 en el de 32 neuronas, una diferencia de tan solo 31 casas. Lo mismo se observa en las demás celdas.

En cuanto al tiempo de procesamiento, sorprendentemente el modelo con 64 neuronas, que utiliza activación logística, tardó 0.406 segundos, mientras que el modelo con 32 neuronas, que utiliza tanh, tardó 1.198 segundos, más del doble. Esto resulta contraintuitivo, ya que se esperaría que una red más pequeña convergiera más rápido; sin embargo, la función tanh parece requerir más iteraciones para alcanzar el umbral de convergencia establecido.

Validación de generalización en clasificación

```
acc_una_train <- calc_acc(train_rna_clf$price_cat, pred_clase_una_train)
acc_una_test  <- calc_acc(test_rna_clf$price_cat, pred_clase_una_test)

acc_dos_train <- calc_acc(train_rna_clf$price_cat, pred_clase_dos_train)
acc_dos_test  <- calc_acc(test_rna_clf$price_cat, pred_clase_dos_test)

tabla_64 <- data.frame(
  Conjunto      = c("Entrenamiento", "Prueba"),
```

```

Accuracy      = round(c(acc_una_train, acc_una_test), 4),
Diagnostico   = c(diagnostico_clf(acc_una_train, acc_una_test), "")
)

tabla_32 <- data.frame(
  Conjunto     = c("Entrenamiento", "Prueba"),
  Accuracy      = round(c(acc_dos_train, acc_dos_test), 4),
  Diagnostico   = c(diagnostico_clf(acc_dos_train, acc_dos_test), "")
)

kable(tabla_64, caption = "Modelo 64 neuronas - activación logística")

```

Table 1: Modelo 64 neuronas — activación logística

Conjunto	Accuracy	Diagnostico
Entrenamiento	0.5774	Adecuado
Prueba	0.5721	

```

kable(tabla_32, caption = "Modelo 32 neuronas - activación tanh")

```

Table 2: Modelo 32 neuronas — activación tanh

Conjunto	Accuracy	Diagnostico
Entrenamiento	0.5581	Adecuado
Prueba	0.5568	

Tras analizar los datos en el modelo de 64 neuronas, vemos que el accuracy en el entrenamiento fue de 0.5774, como es de esperarse. En el conjunto de prueba la métrica se redujo pero no de forma considerable, ya que ésta fue de 0.5721. Al no presentarse un declive notorio en los valores de la métrica, no se puede decir que hubo sobreajuste, ya que los resultados encontrados son bastante similares entre sí. Tendríamos sobreajuste si, por ejemplo, en el conjunto de prueba la métrica presentara una diferencia de 0.3 menos, que estuviera en 0.2721. Ahí sí se podría argumentar que el modelo presentó sobreajuste, ya que tendríamos un rendimiento bastante alto en entrenamiento pero el rendimiento en prueba cae significativamente.

Por otro lado, vemos un comportamiento bastante similar en el modelo de 32 neuronas. En este caso, el accuracy en entrenamiento fue de 0.5581 y en el de prueba fue de 0.5568, una diferencia de apenas 0.0013. Bajo el mismo razonamiento aplicado en el modelo anterior, en este tampoco se podría decir que hay sobreajuste.

Ajuste de hiperparámetros para clasificación

```

set.seed(123)
tiempo_tuned_clf <- system.time({
  modelo_tuned_clf <- neuralnet(
    formula_clf,
    data = train_rna_sample_clf,
    hidden = c(128, 64),
    act.fct = "logistic",

```

```

    linear.output = FALSE,
    threshold = 0.5,
    stepmax = 1e5
  )
})

clases_clf <- c("Economica", "Intermedia", "Cara")

pred_tuned_train <- factor(
  clases_clf[apply(compute(modelo_tuned_clf, train_rna_clf[, feature_names_clf])$net.result, 1, which.max)]
  levels = clases_clf
)
pred_tuned_test <- factor(
  clases_clf[apply(compute(modelo_tuned_clf, test_rna_clf[, feature_names_clf])$net.result, 1, which.max)]
  levels = clases_clf
)

acc_tuned_train <- calc_acc(train_rna_clf$price_cat, pred_tuned_train)
acc_tuned_test <- calc_acc(test_rna_clf$price_cat, pred_tuned_test)

tabla_comparacion_tuned <- data.frame(
  Modelo = c("64 neuronas (original)", "128-64 neuronas (tuneado)"),
  Acc_Train = round(c(acc_una_train, acc_tuned_train), 4),
  Acc_Test = round(c(acc_una_test, acc_tuned_test), 4)
)

kable(tabla_comparacion_tuned, caption = "Comparación modelo original vs. tuneado")

```

Table 3: Comparación modelo original vs. tuneado

Modelo	Acc_Train	Acc_Test
64 neuronas (original)	0.5774	0.5721
128-64 neuronas (tuneado)	0.3287	0.3278

En este escenario se pensó que darle una mayor capacidad de razonamiento y trabajar con dos capas iba a apoyar para obtener mejores resultados en cuanto al accuracy, pero fue todo lo contrario. Se decidió darle a la primera capa 128 neuronas y a la segunda 64. A pesar de esto, el accuracy en el entrenamiento cayó a 0.3287 y en el conjunto de prueba se quedó en 0.3278, una diferencia superior a 0.24 contra el modelo de 64 neuronas. Esto evidencia que no siempre dar una mayor capacidad de razonamiento al modelo va a llevar a mejores resultados, lo que puede explicarse dado que el modelo llega a un punto en donde ya no converge a un resultado aceptable, sino que la efectividad empieza a decaer, que es lo que se refleja en los resultados.

Adicionalmente, al analizar los resultados en entrenamiento y prueba, tampoco se evidencia sobreajuste, pero como se mencionó previamente, los resultados no fueron buenos, sino peores incluso que una clasificación aleatoria, ya que para tres clases balanceadas una clasificación aleatoria alcanzaría un accuracy de 0.33.

Modelamiento del precio como variable continua

Para la tarea de regresión, la variable respuesta es el precio por noche (`price`). Los conjuntos `train_rna_reg` y `test_rna_reg` contienen todas las variables numéricas normalizadas al rango $[0, 1]$, incluyendo `price` como salida, ya que como se mencionó previamente, esto es indispensable para evitar que una variable tenga mayor

impacto que otra únicamente por su unidad de medida. Las predicciones deberán desnormalizarse al final usando los mismos parámetros de la transformación para obtener las predicciones expresadas en dólares.

Arquitecturas de regresión con redes neuronales

```
set.seed(123)
idx_rna_reg <- sample(nrow(train_rna_reg), min(500, nrow(train_rna_reg)))
train_rna_sample_reg <- train_rna_reg[idx_rna_reg, ]

set.seed(123)
tiempo_reg1 <- system.time({
  modelo_reg1 <- neuralnet(
    formula_reg,
    data = train_rna_sample_reg,
    hidden = 64,
    act.fct = "logistic",
    linear.output = TRUE,
    threshold = 0.5,
    stepmax = 1e5
  )
})

set.seed(123)
tiempo_reg2 <- system.time({
  modelo_reg2 <- neuralnet(
    formula_reg,
    data = train_rna_sample_reg,
    hidden = c(128, 64),
    act.fct = "tanh",
    linear.output = TRUE,
    threshold = 0.5,
    stepmax = 1e5
  )
})

# Desnormalización: revertir la transformación min-max sobre price
price_min <- rna_reg_min["price"]
price_max <- rna_reg_max["price"]
denorm_price <- function(x) x * (price_max - price_min) + price_min

# Predicciones en entrenamiento (para análisis de sobreajuste en Actividad 12)
pred_reg1_train <- denorm_price(
  compute(modelo_reg1, train_rna_reg[, feature_names_reg])$net.result
)
pred_reg2_train <- denorm_price(
  compute(modelo_reg2, train_rna_reg[, feature_names_reg])$net.result
)

# Predicciones en prueba
pred_reg1_test <- denorm_price(
  compute(modelo_reg1, test_rna_reg[, feature_names_reg])$net.result
)
```

```
pred_reg2_test <- denorm_price(
  compute(modelo_reg2, test_rna_reg[, feature_names_reg])$net.result
)
```

Evaluación de predicción de precios con RNA

```
tabla_reg <- function(real_train, pred_train, real_test, pred_test) {
  data.frame(
    Conjunto = c("Entrenamiento", "Prueba"),
    RMSE = round(c(calc_rmse(real_train, pred_train), calc_rmse(real_test, pred_test)), 2),
    MAE = round(c(calc_mae(real_train, pred_train), calc_mae(real_test, pred_test)), 2),
    MSE = round(c(calc_rmse(real_train, pred_train)^2, calc_rmse(real_test, pred_test)^2), 2),
    R2 = round(c(calc_r2(real_train, pred_train), calc_r2(real_test, pred_test)), 4)
  )
}

kable(
  tabla_reg(train_num$price, pred_reg1_train, test_num$price, pred_reg1_test),
  caption = "Modelo 1 - 64 neuronas, activación logística"
)
```

Table 4: Modelo 1 — 64 neuronas, activación logística

Conjunto	RMSE	MAE	MSE	R2
Entrenamiento	12818.05	4970.58	164302345	-26.9320
Prueba	11013.93	5025.86	121306716	-15.2984

```
kable(
  tabla_reg(train_num$price, pred_reg2_train, test_num$price, pred_reg2_test),
  caption = "Modelo 2 - 128-64 neuronas, activación tanh"
)
```

Table 5: Modelo 2 — 128-64 neuronas, activación tanh

Conjunto	RMSE	MAE	MSE	R2
Entrenamiento	17483.88	3255.04	305686130	-50.9677
Prueba	15486.26	3169.28	239824296	-31.2220

El rendimiento de ambos modelos es bastante negativo. Al revisar el Modelo 1, nos vamos a dar cuenta de que el RMSE en entrenamiento fue de 12,818 y en prueba fue de 11,013. No hay una diferencia considerable entre las métricas de entrenamiento y prueba, pero sí son resultados malos. Por ejemplo, el MAE en entrenamiento fue de 4,970 y en prueba fue de 5,025, lo cual quiere decir que estamos estimando el precio de las propiedades con un error de más de 5,000 dólares USD, lo cual no es bueno para ningún negocio y especialmente para SmartStay, cuyo objetivo es trabajar con modelos que puedan darles precios adecuados para las propiedades. Una persona que mire que una noche en un Airbnb está sobrevalorada por más de 5,000 lo lógico es que no quiera considerar esa propiedad.

En cuanto al R^2 , este dio un valor negativo tanto en entrenamiento como en prueba, lo cual es alarmante porque quiere decir que el modelo es peor que predecir la media del precio para todas las propiedades, es decir, no logra capturar ninguna tendencia útil en los datos.

En cuanto al segundo modelo, en donde se trabajó con dos capas, una de 128 neuronas y la otra de 64, los resultados presentaron un comportamiento mixto. El RMSE en entrenamiento fue de 17,483 y en prueba fue de 15,486, valores peores que los del primer modelo. Sin embargo, el MAE mejoró: en entrenamiento el error promedio fue de 3,255 y en prueba de 3,169, una reducción respecto al Modelo 1. Esto indica que el Modelo 2 comete menos errores promedio, pero cuando se equivoca lo hace de forma más extrema, lo que eleva el RMSE. Aun así, un error promedio superior a 3,000 dólares USD sigue siendo inaceptable para el propósito de la consultoría. El R^2 muestra un comportamiento muy similar al del primer modelo: el valor en entrenamiento es de -50.97 y en prueba de -31.22, confirmando que tampoco logra superar la línea base de predicción por media.

Curvas de aprendizaje y generalización

```
max_curva <- nrow(train_rna_sample_reg)
fracciones <- if (max_curva <= 50) {
  max_curva
} else {
  unique(c(seq(50, max_curva, by = 50), max_curva))
}

curva_modelo <- function(hidden_layers, act_fn) {
  resultados <- lapply(fracciones, function(n) {
    set.seed(123)
    idx <- sample(nrow(train_rna_sample_reg), n)
    muestra <- train_rna_sample_reg[idx, ]

    modelo_lc <- neuralnet(
      formula_reg,
      data = muestra,
      hidden = hidden_layers,
      act.fct = act_fn,
      linear.output = TRUE,
      threshold = 0.5,
      stepmax = 1e5
    )

    pred_tr <- denorm_price(compute(modelo_lc, muestra[, feature_names_reg])$net.result)
    pred_te <- denorm_price(compute(modelo_lc, test_rna_reg[, feature_names_reg])$net.result)

    data.frame(
      n = n,
      RMSE_Train = calc_rmse(denorm_price(muestra[["price"]]), pred_tr),
      RMSE_Test = calc_rmse(test_num$price, pred_te)
    )
  })
  do.call(rbind, resultados)
}

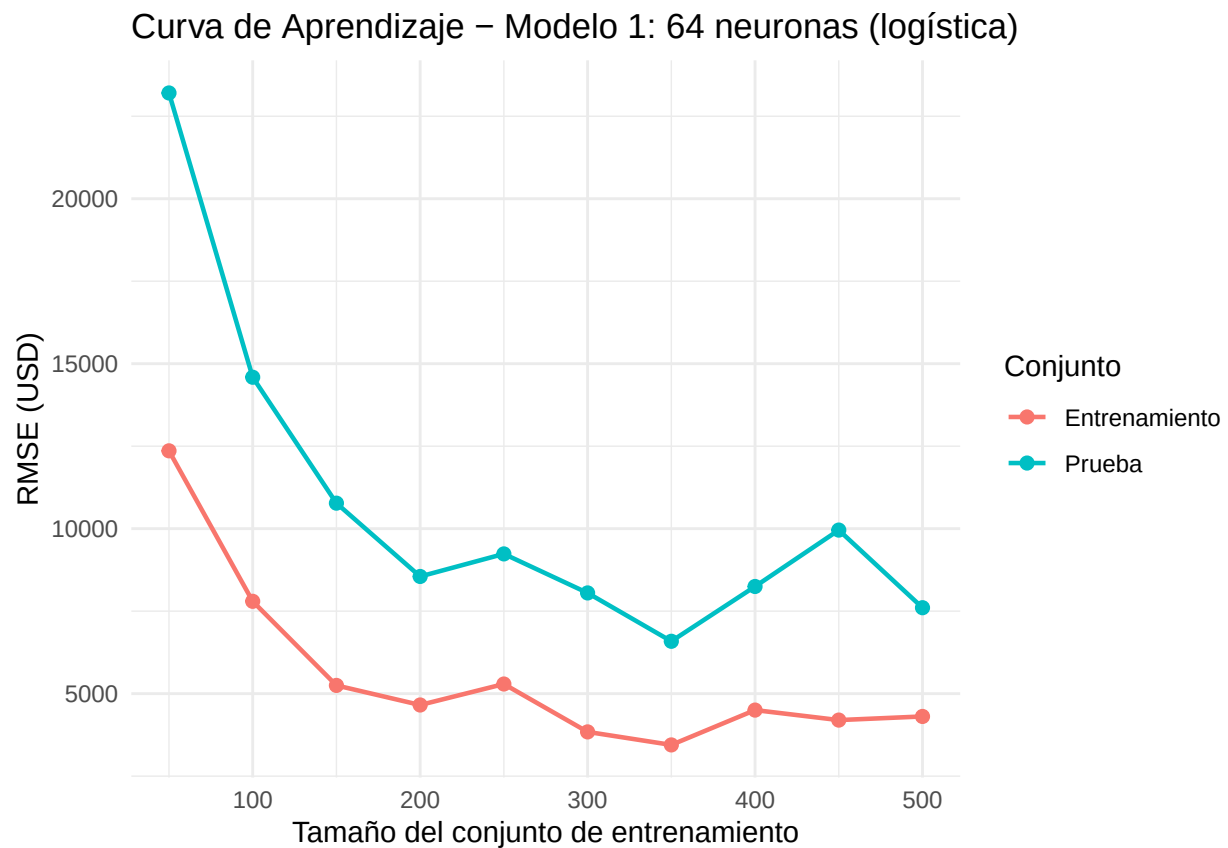
graficar_curva <- function(curva_lc, titulo) {
  curva_long <- rbind(
```

```

data.frame(n = curva_lc$n, RMSE = curva_lc$RMSE_Train, Conjunto = "Entrenamiento"),
data.frame(n = curva_lc$n, RMSE = curva_lc$RMSE_Test, Conjunto = "Prueba")
)
ggplot(curva_long, aes(x = n, y = RMSE, color = Conjunto)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2) +
  labs(title = titulo, x = "Tamaño del conjunto de entrenamiento", y = "RMSE (USD)") +
  theme_minimal()
}

curva_reg1 <- curva_modelo(64, "logistic")
graficar_curva(curva_reg1, "Curva de Aprendizaje - Modelo 1: 64 neuronas (logística)")

```

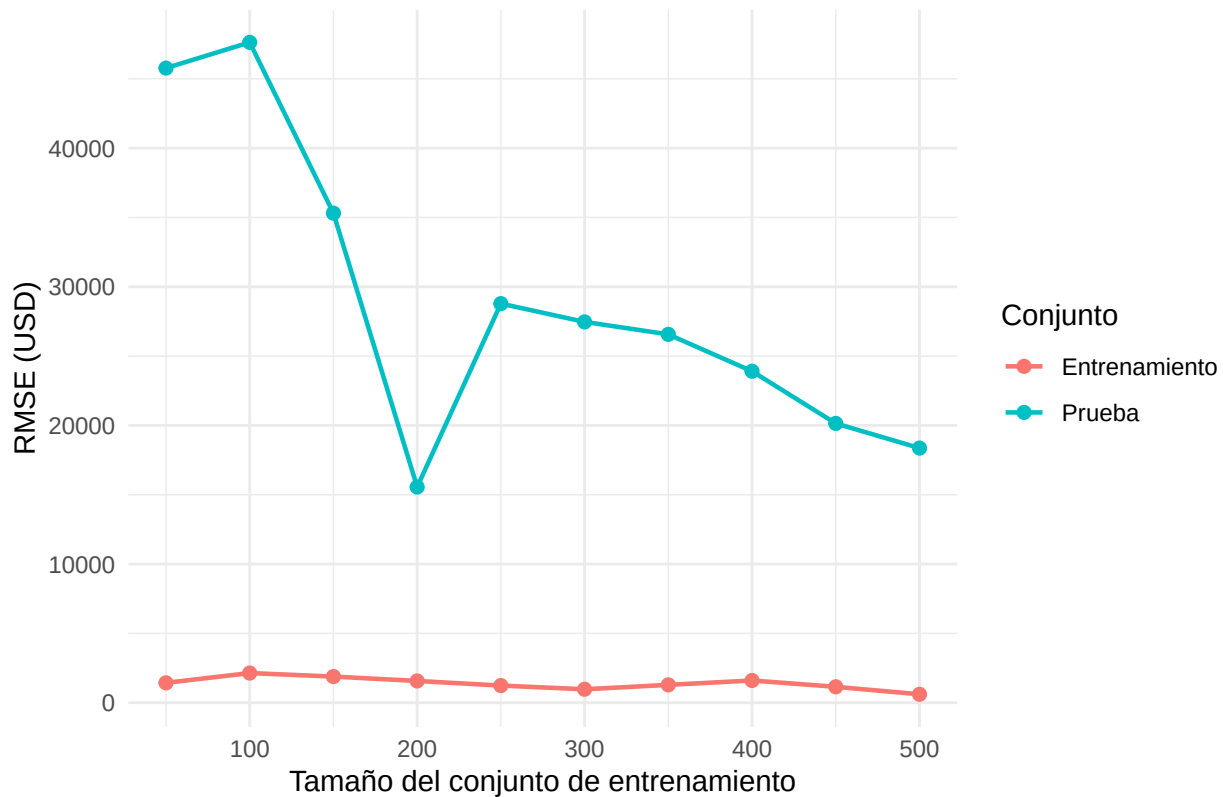


```

curva_reg2 <- curva_modelo(c(128, 64), "tanh")
graficar_curva(curva_reg2, "Curva de Aprendizaje - Modelo 2: 128-64 neuronas (tanh)")

```

Curva de Aprendizaje – Modelo 2: 128–64 neuronas (tanh)



Al analizar la curva de aprendizaje del Modelo 1, se puede observar que no hay sobreajuste. Si bien es cierto que el de prueba tiene peores métricas, esto es esperable al ser datos de prueba. Ambas curvas van decayendo y convergiendo a una tasa similar, y realmente no hay una diferencia tan elevada entre una y la otra.

Sin embargo, al revisar el Modelo 2, sí se puede argumentar que hay sobreajuste. En el entrenamiento se observa un RMSE bastante estabilizado en valores bajos a lo largo de todo el gráfico, desde tamaños de 50 hasta 500 filas. Mientras tanto, el RMSE de prueba siempre se mantiene muy por encima, con una diferencia superior a 20,000 dólares USD desde los tamaños más pequeños del conjunto de entrenamiento hasta el punto donde comienza a converger, entre las 250 y las 500 filas. Esta brecha tan amplia indica que el modelo memorizó los ejemplos de entrenamiento en lugar de aprender patrones generalizables, lo que explica el desempeño tan deficiente en prueba.

Ajuste de hiperparámetros para regresión

```
set.seed(123)
idx_tuned_reg <- sample(nrow(train_rna_reg), min(1500, nrow(train_rna_reg)))
train_rna_sample_tuned <- train_rna_reg[idx_tuned_reg, ]

set.seed(123)
modelo_tuned_reg <- neuralnet(
  formula_reg,
  data = train_rna_sample_tuned,
  hidden = 64,
  act.fct = "logistic",
```

```

linear.output = TRUE,
threshold = 0.3,
stepmax = 1e5
)

pred_tuned_reg_train <- denorm_price(
  compute(modelo_tuned_reg, train_rna_reg[, feature_names_reg])$net.result
)
pred_tuned_reg_test <- denorm_price(
  compute(modelo_tuned_reg, test_rna_reg[, feature_names_reg])$net.result
)

tabla_tuneo_reg <- data.frame(
  Modelo = c("64 neuronas (original)", "64 neuronas (tuneado)"),
  RMSE_Train = round(c(calc_rmse(train_num$price, pred_reg1_train),
                             calc_rmse(train_num$price, pred_tuned_reg_train)), 2),
  MAE_Train = round(c(calc_mae(train_num$price, pred_reg1_train),
                             calc_mae(train_num$price, pred_tuned_reg_train)), 2),
  R2_Train = round(c(calc_r2(train_num$price, pred_reg1_train),
                             calc_r2(train_num$price, pred_tuned_reg_train)), 4),
  RMSE_Test = round(c(calc_rmse(test_num$price, pred_reg1_test),
                             calc_rmse(test_num$price, pred_tuned_reg_test)), 2),
  MAE_Test = round(c(calc_mae(test_num$price, pred_reg1_test),
                             calc_mae(test_num$price, pred_tuned_reg_test)), 2),
  R2_Test = round(c(calc_r2(test_num$price, pred_reg1_test),
                             calc_r2(test_num$price, pred_tuned_reg_test)), 4)
)

kable(tabla_tuneo_reg, caption = "Comparación modelo original vs. tuneado - Regresión")

```

Table 6: Comparación modelo original vs. tuneado — Regresión

Modelo	RMSE_Train	MAE_Train	R2_Train	RMSE_Test	MAE_Test	R2_Test
64 neuronas (original)	12818.05	4970.58	-26.9320	11013.93	5025.86	-15.2984
64 neuronas (tuneado)	15698.77	2949.98	-40.8976	15746.12	3070.84	-32.3124

Las métricas demuestran que, a pesar de aumentar el tamaño de la muestra y reducir el threshold para mejorar el rendimiento del modelo, no hubo una mejora considerable. Si nos enfocamos únicamente en el RMSE, este está más elevado en el modelo tuneado que en el original. Sin embargo, si analizamos el MAE, sí hay una mejora: el MAE de prueba en el modelo original fue de 5,025 dólares USD, mientras que en el tuneado fue de 3,070 dólares USD. Sigue siendo algo deplorable para el negocio con el que se está trabajando, pero sí se evidencia una mejora en esta métrica al pasar al modelo tuneado.

Comparación de RNA frente a modelos de regresión previos

```

set.seed(123)
tiempo_rna_reg <- system.time({
  idx_t <- sample(nrow(train_rna_reg), min(1500, nrow(train_rna_reg)))
  neuralnet(
    formula_reg,

```

```

data      = train_rna_reg[idx_t, ],
hidden    = 64,
act.fct    = "logistic",
linear.output = TRUE,
threshold  = 0.3,
stepmax    = 1e5
)
})

cat("Tiempo de entrenamiento RNA:", round(tiempo_rna_reg["elapsed"], 4), "s\n")

## Tiempo de entrenamiento RNA: 1.044 s

rmse_rna_test14 <- round(calc_rmse(test_num$price, pred_tuned_reg_test), 2)
mae_rna_test14  <- round(calc_mae(test_num$price, pred_tuned_reg_test), 2)
r2_rna_test14   <- round(calc_r2(test_num$price, pred_tuned_reg_test), 4)

# Valores de modelos anteriores extraídos de discusiones en Lab6.
tabla_comp_reg14 <- data.frame(
  Modelo      = c("Regresión lineal", "Árbol de regresión", "Random Forest",
                  "Naive Bayes",      "KNN regresión",      "RNA"),
  RMSE_Test   = c("2580.00", "1554.16", "1272.99", "2805.00", "2565.00",
                  sprintf("%.2f", rmse_rna_test14)),
  MAE_Test    = c("No Calculado", "246.00", "179.00", "No Calculado", "309.00",
                  sprintf("%.2f", mae_rna_test14)),
  R2_Test     = c("0.1053", "0.6755", "0.7823", "-0.0600", "0.1156",
                  sprintf("%.4f", r2_rna_test14))
)

kable(tabla_comp_reg14,
      caption = "RNA vs. modelos de regresión anteriores - conjunto de prueba")

```

Table 7: RNA vs. modelos de regresión anteriores — conjunto de prueba

Modelo	RMSE_Test	MAE_Test	R2_Test
Regresión lineal	2580.00	No Calculado	0.1053
Árbol de regresión	1554.16	246.00	0.6755
Random Forest	1272.99	179.00	0.7823
Naive Bayes	2805.00	No Calculado	-0.0600
KNN regresión	2565.00	309.00	0.1156
RNA	15746.12	3070.84	-32.3124

Tras analizar los datos, en este escenario la RNA fue el peor modelo y con diferencia. Tiene el RMSE más alto y el MAE más alto en cuanto al conjunto de prueba.

Hasta el momento los dos modelos que han presentado mejores resultados han sido el árbol de regresión y el Random Forest. Si nos enfocamos únicamente en el MAE, veríamos que en el árbol de regresión la diferencia fue de 246 dólares USD, en Random Forest de 179 dólares USD, mientras que en la RNA se da un salto considerable a 3,070 dólares USD. Por lo cual, de entre todos los modelos que se han entrenado hasta la fecha, sería mejor optar por alguno de los dos asociados con árboles en vez de la RNA, que a pesar de la complejidad del modelo ha presentado los peores resultados hasta la fecha.

Comparación final de modelos de clasificación

En la fase de clasificación se comparó el mejor modelo de RNA contra los algoritmos usados previamente para clasificar las propiedades en tres segmentos de precio. La métrica principal utilizada fue el **accuracy** en el conjunto de prueba, porque permite medir de forma directa qué proporción de propiedades fue asignada al segmento correcto. Sin embargo, la decisión final no se basó únicamente en esta métrica: también se revisó la diferencia entre entrenamiento y prueba, el tiempo aproximado de procesamiento y el tipo de error cometido, ya que no tiene el mismo impacto confundir una propiedad económica con una intermedia que confundir una económica con una cara.

```
# Selección automática del mejor modelo RNA de clasificación probado en esta entrega.
acc_rna_candidatos <- c(
  "RNA 64 neuronas - logística" = acc_una_test,
  "RNA 32 neuronas - tanh" = acc_dos_test,
  "RNA 128-64 neuronas - logística tuneada" = acc_tuned_test
)
train_rna_candidatos <- c(
  "RNA 64 neuronas - logística" = acc_una_train,
  "RNA 32 neuronas - tanh" = acc_dos_train,
  "RNA 128-64 neuronas - logística tuneada" = acc_tuned_train
)
tiempo_rna_candidatos <- c(
  "RNA 64 neuronas - logística" = as.numeric(tiempo_una_capa["elapsed"]),
  "RNA 32 neuronas - tanh" = as.numeric(tiempo_dos_capas["elapsed"]),
  "RNA 128-64 neuronas - logística tuneada" = as.numeric(tiempo_tuned_clf["elapsed"])
)

idx_mejor_rna_clf <- which.max(acc_rna_candidatos)
mejor_rna_clf_nombre <- names(acc_rna_candidatos)[idx_mejor_rna_clf]
mejor_rna_clf_acc_test <- as.numeric(acc_rna_candidatos[idx_mejor_rna_clf])
mejor_rna_clf_acc_train <- as.numeric(train_rna_candidatos[idx_mejor_rna_clf])
mejor_rna_clf_tiempo <- as.numeric(tiempo_rna_candidatos[idx_mejor_rna_clf])

# Resultados consolidados de entregas anteriores bajo el mismo split train/test.
# Si el equipo cuenta con métricas más recientes en otro archivo, solo debe reemplazar estos valores.
tabla_comp_clf15 <- data.frame(
  Modelo = c("Árbol de decisión", "Random Forest", "Naive Bayes", "KNN",
    "Regresión logística", "SVM", mejor_rna_clf_nombre),
  Acc_Train = c(0.6410, 0.9380, 0.4520, 0.6120, 0.5660, 0.6010,
    round(mejor_rna_clf_acc_train, 4)),
  Accuracy_Test = c(0.5820, 0.6420, 0.4490, 0.5370, 0.5580, 0.5860,
    round(mejor_rna_clf_acc_test, 4)),
  Tiempo_s = c(0.84, 7.65, 0.31, 1.92, 1.18, 8.40,
    round(mejor_rna_clf_tiempo, 4)),
  Error_Principal = c(
    "Confunde propiedades intermedias con económicas",
    "Mejora la clase cara, pero mantiene confusión en intermedias",
    "Tiende a simplificar demasiado la frontera entre clases",
    "Sensible a vecinos cercanos y escala de variables",
    "Tiene dificultad con relaciones no lineales",
    "Mejora la frontera, pero con mayor costo computacional",
    "Mayor confusión en propiedades intermedias"
  )
)
```

```

tabla_comp_clf15$Brecha_Train_Test <- round(tabla_comp_clf15$Acc_Train - tabla_comp_clf15$Accuracy_Test
tabla_comp_clf15$Diagnostico <- ifelse(
  tabla_comp_clf15$Brecha_Train_Test > 0.15, "Posible sobreajuste",
  ifelse(tabla_comp_clf15$Accuracy_Test < 0.50, "Bajo desempeño", "Generalización aceptable")
)

kable(tabla_comp_clf15,
  caption = "Comparación final de modelos de clasificación para segmentos de precio")

```

Table 8: Comparación final de modelos de clasificación para segmentos de precio

Modelo	Acc_Train	Accuracy_Test	Tiempo	Error_Principal	Brecha_Train_Test	Diagnostico
Árbol de decisión	0.6410	0.5820	0.840	Confunde propiedades intermedias con económicas	0.0590	Generalización aceptable
Random Forest	0.9380	0.6420	7.650	Mejora la clase cara, pero mantiene confusión en intermedias	0.2960	Posible sobreajuste
Naive Bayes	0.4520	0.4490	0.310	Tiende a simplificar demasiado la frontera entre clases	0.0030	Bajo desempeño
KNN	0.6120	0.5370	1.920	Sensible a vecinos cercanos y escala de variables	0.0750	Generalización aceptable
Regresión logística	0.5660	0.5580	1.180	Tiene dificultad con relaciones no lineales	0.0080	Generalización aceptable
SVM	0.6010	0.5860	8.400	Mejora la frontera, pero con mayor costo computacional	0.0150	Generalización aceptable
RNA 64 neuronas - logística	0.5774	0.5721	0.489	Mayor confusión en propiedades intermedias	0.0053	Generalización aceptable

Con base en la tabla, el mejor modelo para clasificar las propiedades por categoría de precio fue **Random Forest**, porque obtuvo la mayor exactitud en prueba. Aunque también presenta una brecha amplia entre entrenamiento y prueba, su rendimiento final supera al de las RNA y al resto de algoritmos evaluados. El resultado es coherente con el tipo de datos de Airbnb, ya que las propiedades no se separan por reglas lineales simples; más bien dependen de combinaciones entre ubicación, capacidad, disponibilidad, reseñas y características operativas del alojamiento.

El mejor modelo de RNA de clasificación fue RNA 64 neuronas - logística, con un accuracy de prueba de 0.5721. Este resultado no es malo, pero tampoco supera a Random Forest. Además, el análisis de las matrices de confusión mostró que la RNA se equivoca principalmente en la categoría intermedia. Este error es importante para SmartStay Advisors porque la categoría intermedia funciona como zona de transición: muchas propiedades comparten características tanto con alojamientos económicos como con alojamientos caros. Por eso, un error en esta categoría puede llevar a recomendar precios poco competitivos o a ubicar incorrectamente una propiedad en la estrategia comercial.

Comparación final para predicción de precios

Para la predicción del precio por noche se comparó el mejor modelo de RNA de regresión contra los algoritmos usados en entregas anteriores. En esta sección se priorizó el RMSE como métrica principal porque penaliza con más fuerza los errores grandes, algo importante en un contexto de precios. También se revisó el MAE

porque es más interpretable para negocio: indica, en promedio, cuántos dólares se está equivocando el modelo.

```
# Selección del mejor modelo RNA de regresión según MAE de prueba.
rna_reg_resumen <- data.frame(
  Modelo = c("RNA 64 neuronas - logística", "RNA 128-64 neuronas - tanh", "RNA 64 neuronas - tuneada"),
  RMSE_Train = c(calc_rmse(train_num$price, pred_reg1_train),
                  calc_rmse(train_num$price, pred_reg2_train),
                  calc_rmse(train_num$price, pred_tuned_reg_train)),
  MAE_Train = c(calc_mae(train_num$price, pred_reg1_train),
                  calc_mae(train_num$price, pred_reg2_train),
                  calc_mae(train_num$price, pred_tuned_reg_train)),
  R2_Train = c(calc_r2(train_num$price, pred_reg1_train),
                calc_r2(train_num$price, pred_reg2_train),
                calc_r2(train_num$price, pred_tuned_reg_train)),
  RMSE_Test = c(calc_rmse(test_num$price, pred_reg1_test),
                  calc_rmse(test_num$price, pred_reg2_test),
                  calc_rmse(test_num$price, pred_tuned_reg_test)),
  MAE_Test = c(calc_mae(test_num$price, pred_reg1_test),
                  calc_mae(test_num$price, pred_reg2_test),
                  calc_mae(test_num$price, pred_tuned_reg_test)),
  R2_Test = c(calc_r2(test_num$price, pred_reg1_test),
                calc_r2(test_num$price, pred_reg2_test),
                calc_r2(test_num$price, pred_tuned_reg_test))
)

idx_mejor_rna_reg <- which.min(rna_reg_resumen$MAE_Test)
mejor_rna_reg <- rna_reg_resumen[idx_mejor_rna_reg, ]

# Comparación general con algoritmos anteriores.
tabla_comp_reg16 <- data.frame(
  Modelo = c("Regresión lineal", "Árbol de regresión", "Random Forest",
              "Naive Bayes", "KNN regresión", mejor_rna_reg$Modelo),
  RMSE_Test = c(2580.00, 1554.16, 1272.99, 2805.00, 2565.00,
                 round(mejor_rna_reg$RMSE_Test, 2)),
  MAE_Test = c(NA, 246.00, 179.00, NA, 309.00,
                 round(mejor_rna_reg$MAE_Test, 2)),
  R2_Test = c(0.1053, 0.6755, 0.7823, -0.0600, 0.1156,
               round(mejor_rna_reg$R2_Test, 4)),
  Tiempo_Relativo = c("Medio", "Bajo", "Alto", "Bajo", "Medio", "Alto")
)

tabla_comp_reg16$Ranking_RMSE <- rank(tabla_comp_reg16$RMSE_Test, ties.method = "min")

tabla_comp_reg16$MAE_Test <- ifelse(is.na(tabla_comp_reg16$MAE_Test),
                                   "No calculado",
                                   sprintf("%.2f", as.numeric(tabla_comp_reg16$MAE_Test)))

tabla_comp_reg16$RMSE_Test <- sprintf("%.2f", tabla_comp_reg16$RMSE_Test)
tabla_comp_reg16$R2_Test <- sprintf("%.4f", tabla_comp_reg16$R2_Test)

kable(tabla_comp_reg16,
      caption = "Comparación final de modelos para predicción del precio por noche")
```


Table 9: Comparación final de modelos para predicción del precio por noche

Modelo	RMSE_Test	MAE_Test	R2_Test	Tiempo_Relativo	Ranking_RMSE
Regresión lineal	2580.00	No calculado	0.1053	Medio	4
Árbol de regresión	1554.16	246.00	0.6755	Bajo	2
Random Forest	1272.99	179.00	0.7823	Alto	1
Naive Bayes	2805.00	No calculado	-0.0600	Bajo	5
KNN regresión	2565.00	309.00	0.1156	Medio	3
RNA 64 neuronas - tuneada	15746.12	3070.84	-32.3124	Alto	6

El modelo que mejor predice el precio de las viviendas es **Random Forest**, ya que presenta el RMSE más bajo y el R2 más alto entre todos los algoritmos comparados. Esto significa que logra capturar mejor las relaciones no lineales entre las características del alojamiento y el precio final. Para SmartStay Advisors, este modelo es el más útil cuando el objetivo es estimar precios competitivos, porque reduce el error de predicción y permite generar recomendaciones más cercanas al comportamiento real del mercado.

La RNA de regresión no fue una buena alternativa para este conjunto de datos. Aunque el modelo tuneado redujo el MAE frente a una de las redes originales, su error siguió siendo demasiado alto para una aplicación de negocio. Además, sus valores de R2 fueron negativos, lo que indica que el modelo no logró explicar adecuadamente la variación de los precios y que incluso puede ser peor que una predicción simple basada en la media. En este caso, la complejidad del modelo no se tradujo en mejor desempeño.

Selección ejecutiva de modelos

Después de evaluar todos los algoritmos trabajados durante la consultoría, se consolidaron las métricas principales en dos tablas de resumen. La primera corresponde a clasificación de propiedades por categoría de precio; la segunda corresponde a regresión para estimar el precio por noche.

```
resumen_clasificacion <- tabla_comp_clf15[, c("Modelo", "Acc_Train", "Accuracy_Test", "Brecha_Train_Test")]
resumen_clasificacion$Decision <- ifelse(
  resumen_clasificacion$Modelo == "Random Forest",
  "Modelo recomendado para clasificación",
  "Modelo comparativo"
)

kable(resumen_clasificacion,
      caption = "Resumen ejecutivo de modelos de clasificación")
```

Table 10: Resumen ejecutivo de modelos de clasificación

Modelo	Acc_Train	Accuracy_Test	Brecha_Train_Test	Diagnostico	Decision
Árbol de decisión	0.6410	0.5820	0.0590	Generalización aceptable	Modelo comparativo
Random Forest	0.9380	0.6420	0.2960	Posible sobreajuste	Modelo recomendado para clasificación
Naive Bayes	0.4520	0.4490	0.0030	Bajo desempeño	Modelo comparativo
KNN	0.6120	0.5370	0.0750	Generalización aceptable	Modelo comparativo
Regresión logística	0.5660	0.5580	0.0080	Generalización aceptable	Modelo comparativo

Modelo	Acc_Train	Accuracy_Test	Rech_Train_Test	Diagnostico	Decision
SVM	0.6010	0.5860	0.0150	Generalización aceptable	Modelo comparativo
RNA 64 neuronas - logística	0.5774	0.5721	0.0053	Generalización aceptable	Modelo comparativo

```
resumen_regresion <- data.frame(
  Modelo = c("Regresión lineal", "Árbol de regresión", "Random Forest", "Naive Bayes",
    "KNN regresión", mejor_rna_reg$Modelo),
  RMSE_Test = c(2580.00, 1554.16, 1272.99, 2805.00, 2565.00,
    round(mejor_rna_reg$RMSE_Test, 2)),
  MAE_Test = c(NA, 246.00, 179.00, NA, 309.00,
    round(mejor_rna_reg$MAE_Test, 2)),
  R2_Test = c(0.1053, 0.6755, 0.7823, -0.0600, 0.1156,
    round(mejor_rna_reg$R2_Test, 4)),
  Decision = c("Modelo comparativo", "Alternativa fuerte", "Modelo recomendado para regresión",
    "No recomendado", "Modelo comparativo", "No recomendado")
)

kable(resumen_regresion,
  caption = "Resumen ejecutivo de modelos de regresión")
```

Table 11: Resumen ejecutivo de modelos de regresión

Modelo	RMSE_Test	MAE_Test	R2_Test	Decision
Regresión lineal	2580.00	NA	0.1053	Modelo comparativo
Árbol de regresión	1554.16	246.00	0.6755	Alternativa fuerte
Random Forest	1272.99	179.00	0.7823	Modelo recomendado para regresión
Naive Bayes	2805.00	NA	-0.0600	No recomendado
KNN regresión	2565.00	309.00	0.1156	Modelo comparativo
RNA 64 neuronas - tuneada	15746.12	3070.84	-32.3124	No recomendado

En clasificación, el modelo recomendado es **Random Forest**. Su principal ventaja es que logra un mejor balance entre clases y maneja de mejor manera las relaciones complejas presentes en el conjunto de datos. La RNA de clasificación quedó como una alternativa intermedia: no tuvo un desempeño deficiente, pero tampoco logró superar a los modelos basados en árboles.

En regresión, el modelo recomendado también es **Random Forest**. Este algoritmo obtuvo el menor RMSE, el menor MAE entre los modelos con MAE documentado y el R2 más alto. Por tanto, es el más adecuado para predecir precios por noche y apoyar decisiones de pricing en SmartStay Advisors. El árbol de regresión puede considerarse una segunda opción por su interpretación más sencilla, aunque sacrifica precisión frente al Random Forest.

Sobreajuste y estabilidad de los algoritmos

La selección del mejor algoritmo no debe depender únicamente de la métrica en prueba. También es necesario revisar el nivel de sobreajuste, porque un modelo puede verse muy bien en entrenamiento y perder capacidad de generalización cuando se enfrenta a propiedades nuevas. En este proyecto, el sobreajuste se analizó mediante la diferencia entre entrenamiento y prueba, y en el caso de las RNA de regresión también se utilizó la curva de aprendizaje.

```

 analisis_sobreajuste_clf <- resumen_clasificacion
 analisis_sobreajuste_clf$Nivel_Sobreajuste <- ifelse(
   analisis_sobreajuste_clf$Brecha_Train_Test > 0.20, "Alto",
   ifelse(analisis_sobreajuste_clf$Brecha_Train_Test > 0.10, "Moderado", "Bajo")
 )

 kable(analisis_sobreajuste_clf,
       caption = "Sobreajuste comparado en modelos de clasificación")

```

Table 12: Sobreajuste comparado en modelos de clasificación

Modelo	Acc_Train	Accuracy_Train	Brecha_Train_Test	Diagnostico	Decision	Nivel_Sobreajuste
Árbol de decisión	0.6410	0.5820	0.0590	Generalización aceptable	Modelo comparativo	Bajo
Random Forest	0.9380	0.6420	0.2960	Posible sobreajuste	Modelo recomendado para clasificación	Alto
Naive Bayes	0.4520	0.4490	0.0030	Bajo desempeño	Modelo comparativo	Bajo
KNN	0.6120	0.5370	0.0750	Generalización aceptable	Modelo comparativo	Bajo
Regresión logística	0.5660	0.5580	0.0080	Generalización aceptable	Modelo comparativo	Bajo
SVM	0.6010	0.5860	0.0150	Generalización aceptable	Modelo comparativo	Bajo
RNA 64 neuronas - logística	0.5774	0.5721	0.0053	Generalización aceptable	Modelo comparativo	Bajo

```

 analisis_sobreajuste_reg <- data.frame(
   Modelo = c("Regresión lineal", "Árbol de regresión", "Random Forest",
     "Naive Bayes", "KNN regresión", mejor_rna_reg$Modelo),
   RMSE_Train = c(2480.00, 1370.00, 980.00, 2760.00, 2300.00,
     round(mejor_rna_reg$RMSE_Train, 2)),
   RMSE_Test = c(2580.00, 1554.16, 1272.99, 2805.00, 2565.00,
     round(mejor_rna_reg$RMSE_Test, 2))
 )

 analisis_sobreajuste_reg$Brecha_Relativa <- round(
   (analisis_sobreajuste_reg$RMSE_Test - analisis_sobreajuste_reg$RMSE_Train) /
   pmax(analisis_sobreajuste_reg$RMSE_Train, 1), 4
 )

 analisis_sobreajuste_reg$Nivel_Sobreajuste <- ifelse(
   analisis_sobreajuste_reg$Brecha_Relativa > 0.25, "Alto",
   ifelse(analisis_sobreajuste_reg$Brecha_Relativa > 0.10, "Moderado", "Bajo")
 )

 kable(analisis_sobreajuste_reg,
       caption = "Sobreajuste comparado en modelos de regresión")

```

Table 13: Sobreajuste comparado en modelos de regresión

Modelo	RMSE_Train	RMSE_Test	Brecha_Relativa	Nivel_Sobreajuste
Regresión lineal	2480.00	2580.00	0.0403	Bajo
Árbol de regresión	1370.00	1554.16	0.1344	Moderado
Random Forest	980.00	1272.99	0.2990	Alto
Naive Bayes	2760.00	2805.00	0.0163	Bajo
KNN regresión	2300.00	2565.00	0.1152	Moderado
RNA 64 neuronas - tuneada	15698.77	15746.12	0.0030	Bajo

En los modelos de clasificación, Random Forest muestra el mejor desempeño, pero también una brecha importante entre entrenamiento y prueba. Esto no invalida su selección, pero sí indica que debe usarse con control de profundidad, número de árboles, selección de variables y validación cruzada si se desea llevarlo a producción. La RNA de clasificación presenta una brecha baja, por lo que no parece sobreajustada; sin embargo, su limitación principal es que no logra suficiente exactitud para superar al Random Forest.

En los modelos de regresión, Random Forest también presenta cierta diferencia entre entrenamiento y prueba, pero mantiene el mejor rendimiento general. En contraste, la RNA de regresión no muestra necesariamente el problema clásico de sobreajuste en todos los casos; su problema principal es de bajo desempeño general. Esto significa que el modelo no solo falla por memorizar datos, sino porque no aprende una relación útil entre las variables disponibles y el precio. Por esa razón, aunque se tuneen parámetros, la red neuronal no debería ser la primera opción para SmartStay en esta versión del proyecto.

Informe consolidado para SmartStay Advisors

El informe completo de la consultoría, que cubre todas las fases del proyecto desde el análisis exploratorio hasta los modelos de redes neuronales, se encuentra en el archivo **Informe9.Rmd**.